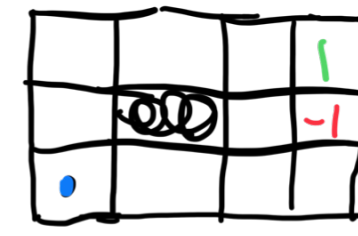


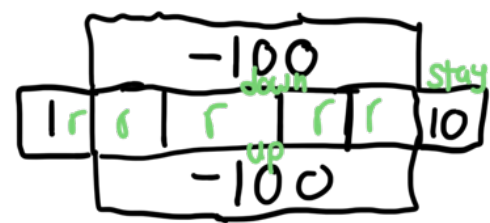
## Markov Decision Process

Collect reward  
Stochastic transitions



$\Pr(s'|s, a)$ : Probability we transition to state  $s'$  if we choose action  $a$  in states  $s$

MDP:  $\Pr(s'|s, a)$ , MP:  $\Pr(s'|s)$



$R(s)$ : Reward function

$\pi(s)$ : Policy (Example)

$U([s_0, s_1, \dots, s_T])$ : Utility of a sequence of states of fixed time

$$= \sum_{t=0}^T R(s_t)$$

$$= \sum_{t=0}^T \gamma^t R(s_t) \quad \gamma \in (0, 1]$$

(Decisions that help u now vs planning for later)

## Value Iteration

This leads to  $U^*(s) = \mathbb{E}_{\Pr(s_0, s_1, \dots, s_T) | s_0=s, \pi} \left[ \sum_{t=0}^T \gamma^t R(s_t) \right]$   $G(\gamma)$  := return, thing u are averaging

Given all future trajectories given a  $s_0$  and  $\pi$ , what is the average reward?

$\pi_s^* = \arg \max_{\pi} U^*(s) = \pi_s^*$  for any  $s'$  **max gives you largest value**

**arg max gives you input that produced largest value**

no one action, but probability of choosing an action

If policy stochastic

$$U^{\pi_s}(s) = \sum_a \pi_s(a) \sum_{s'} P(s'|s, a) [R(s, a, s') + \gamma U^{\pi_s}(s')]$$

Bellman Equation:  $U^*(s) = \max_a \left[ \sum_{s'} P(s'|s, a) (R(s, a, s') + \gamma U^*(s')) \right]$

Recursive def (how it is solved in practice)  $\pi^*(s) = \arg \max_a \left[ \sum_{s'} P(s'|s, a) (R(s, a, s') + \gamma U^*(s')) \right]$  (need  $U^*(s)$  first)

From  $U^*$  u can get  $\pi^*$ , but not vice versa

Initial Guesses for  $\pi_0$  for  $\forall s_0 \in S$   $\pi_0(s_0) = a_0$  deterministic

$$U^{\pi_t}(s) = \sum_{s'} P(s'|s, \pi_t(s)) (R(s, \pi_t(s), s') + \gamma U^{\pi_t}(s'))$$

$$\pi_{t+1}(s) = \arg \max_a \sum_{s'} P(s'|s, a) (R(s, a, s') + \gamma U^{\pi_t}(s'))$$

Value Iteration  
u can say  $s' = s_{t+1}$  &  $s = s_t$  here

- Computes utility for every state
- Needs exact transition model
- Needs fully observe state
- Needs to know exact reward for each state

## Partially-observable MDPs

$(S, A, T, R, \Omega, O, \gamma)$   $a_t$  = action at time  $t$

Hidden  $\left\{ \begin{array}{l} \text{States } S \\ \text{Actions } A \end{array} \right. \quad T: \text{Transition fct: } T(s'|s, a) = P(s_{t+1}=s' | s_t=s, a_t=a)$   
 $R: \text{Reward fct: } R(s, a, s')$

Observable  $\left\{ \begin{array}{l} \text{Observations } \Omega: \text{Sensor readings and tokens the agent can actually perceive} \\ \text{Observation fct } O: O(o|s, a) = P(o_{t+1}=o | s_{t+1}=s', a_t=a) \\ \text{Discount factor } \gamma \end{array} \right.$

agent doesn't know exact state  $s$ , so maintains belief state  $b(s)$

$$\sum_{s \in S} b(s) = 1$$

Belief updates:  $b'(s') = \gamma \cdot O(o|s', a) \sum_{s \in S} T(s'|s, a) b(s)$   $\gamma = \frac{P(o|b, a)}{P(o|b, a)}$  **normalization**

(Bayes rule) Now, we do  $\pi(b(s)) \rightarrow$  action  $a$  instead of  $\pi(s) \rightarrow a$

1. you have  $b_t$

2. Then you have  $a_t = \pi_t(b_t)$

3.  $O_{t+1}$

4.  $b_{t+1}(s') = \gamma O(o_{t+1}|s', a_t) \sum_{s \in S} P(s'|s, a_t) b_t(s)$

5.  $U^{\pi_t}(b_t) = R(b_t, \pi_t(b_t)) + \gamma \sum_{a_{t+1}} P(o_{t+1}|b_t, \pi_t(b_t)) U^{\pi_t}(b_{t+1})$

6.  $\pi_{t+1}(b) = \arg \max_a [R(b_t, a) + \gamma \sum_{o_{t+1}} P(o_{t+1}|b_t, a) U^{\pi_t}(b_{t+1})]$

## Policy Gradient Algorithm & PPO

$$J = \mathbb{E}_{\tau \sim \pi_{\theta}} [R(\tau)] = \int \pi_{\theta}(\tau) R(\tau) d\tau \quad \mathbb{E}[X] = \int x P(x)$$

$$\nabla_{\theta} J = \int R(\tau) [\pi_{\theta} \cdot \nabla_{\theta} \log \pi_{\theta}(\tau)] d\tau = \mathbb{E}_{\tau \sim \pi_{\theta}} (A(\tau) \nabla_{\theta} \log \pi_{\theta}(\tau))$$

Reinforce Alg:

1. Reinforce one full trajectory using  $\pi_{\theta}$
2. Calc  $R(\tau)$  from black box  $\tau$  = trajectory/complete response
3. Calc score:  $\nabla_{\theta} \log \pi_{\theta}(\tau)$
4.  $\theta \leftarrow \theta + \alpha \cdot [R(\tau) \nabla_{\theta} \log \pi_{\theta}(\tau)]$

Now say:  $A(s_t, a_t) = R_t - V(s_t)$

$$\nabla_{\theta} J = \mathbb{E}_{\tau \sim \pi_{\theta}} \left[ \sum_{t=0}^T \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) A(s_t, a_t) \right]$$

Instability tho, i.e massive gradient steps, so we need "safety harness"

## PPO

$$r_t(\theta) = \frac{\pi_{\theta}(a_t | s_t)}{\pi_{\theta_{old}}(a_t | s_t)} \quad L^{clip}(\theta) = \mathbb{E}_t [\min(r_t(\theta) A_t, \text{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon) A_t)]$$

1.  $A_t > 0$   $\min(r_t(\theta) A_t, (1+\epsilon) A_t)$  (Ceiling)  
if  $r_t > 1+\epsilon$  objective goes flat (floor)

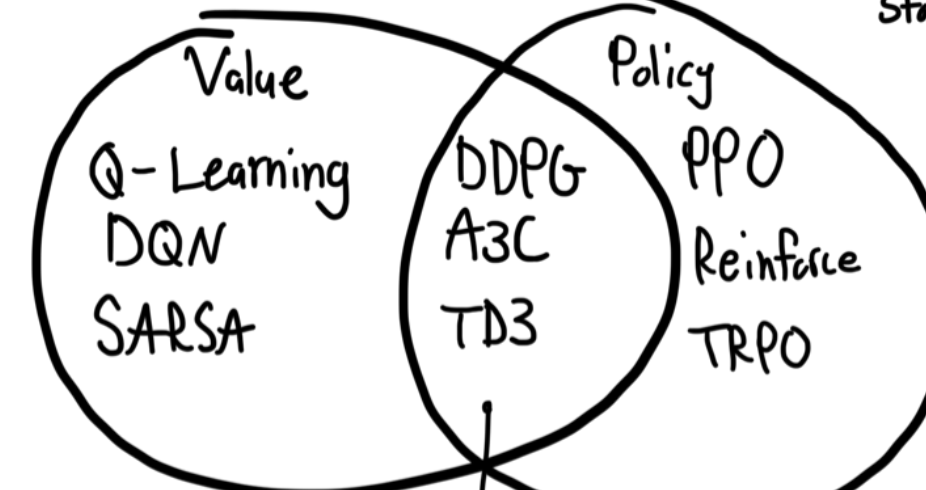
2.  $A_t < 0$ ;  $r_t < 1-\epsilon$

## Actor-Critic Model



Value-based Alg: Learn value fct  $Q(s, a)$ , Implicit  $\pi$ ;  $\arg \max Q$  (Challenges in continuous action space)

Policy-based alg: No value fct learn  $\pi_{\theta}(a|s)$  or  $\mu_{\theta}(a|s)$  (High variance)  
Stochastic vs deterministic



Actor-critic model (best of both worlds)

For recursive stuff they guess, use  $A$  assuming  $B$  is right, then fix  $B$ , etc

TD:  $A$  update,  $B$  update  $\rightarrow A$  update;  $C \rightarrow B \rightarrow A$

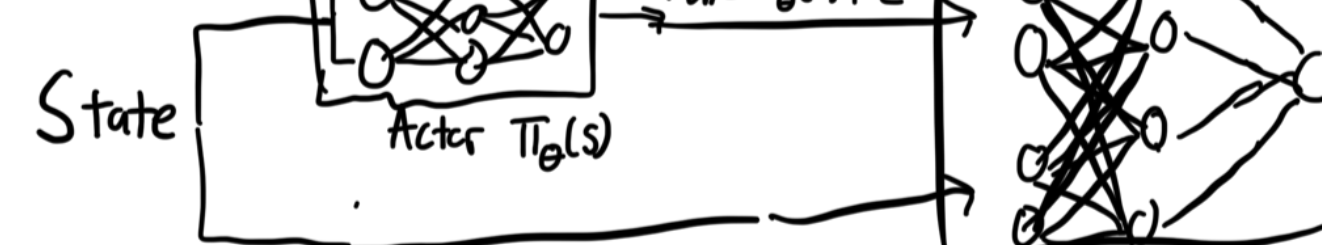
Monte-Carlo: Wait til ending, then go backwards once Gradient

(only if end episode exists/finite)

Actor: Decides which action to take (Policy-based method)

Critic: How good was action & how to adjust (Value-based)

$$Q(s_t, a_t) = \mathbb{E} \left[ \sum_{k=0}^{\infty} \gamma^k R_{t+k} \right]$$



Obtain data experience  $[s_t, a_t, r_t, s_{t+1}]$

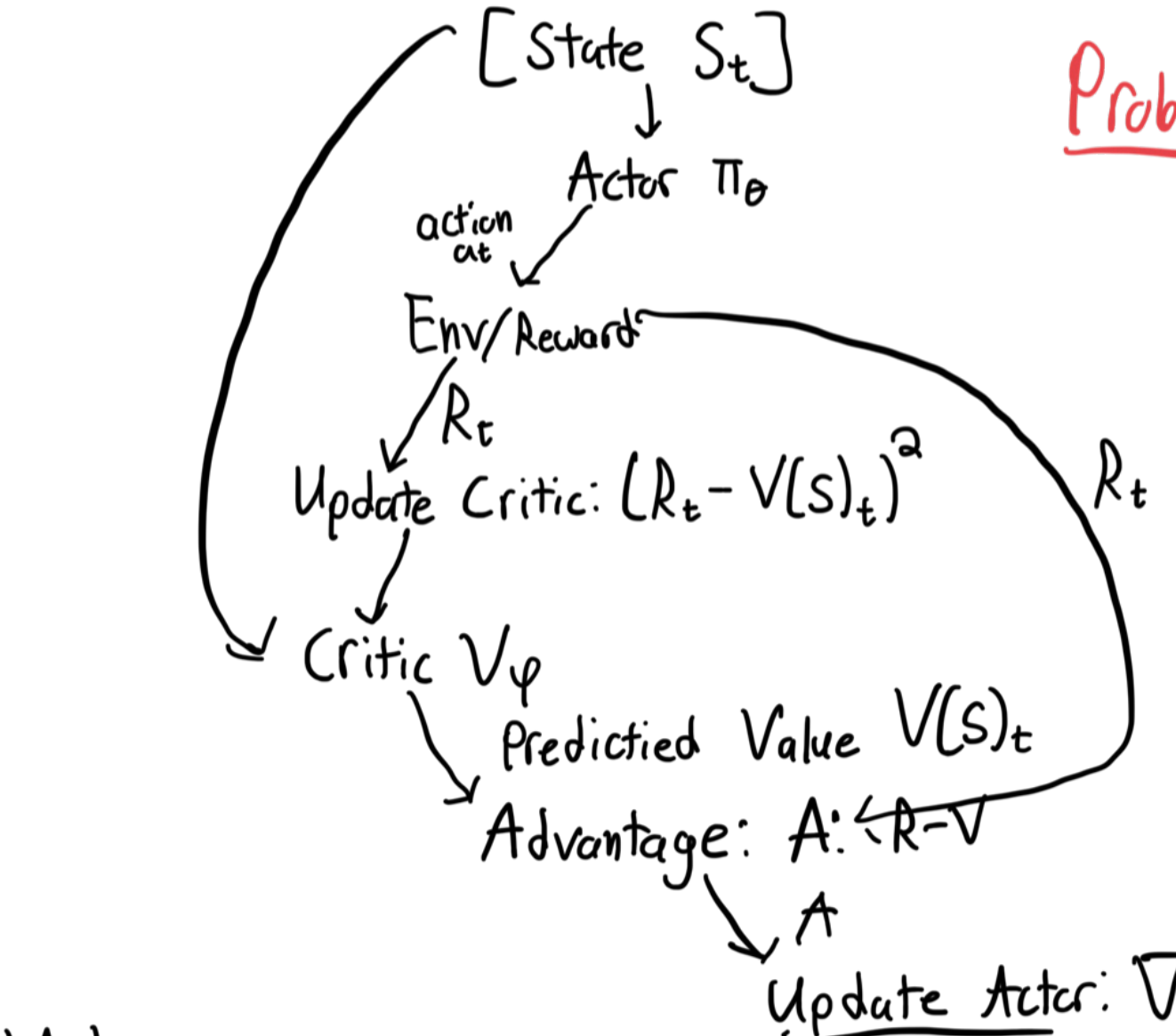
Training

④: Compute  $Q_{\omega}(s_t, a_t)$

⑤  $\Delta \theta = \alpha \nabla_{\theta} \log \pi_{\theta}(s_t, a_t) Q_{\omega}(s_t, a_t)$  **Actor updates policy params  $\theta$  using Q-val  $\rightarrow$  policy grad**

⑥  $\Delta \omega = \beta [R(s_t, a_t) + \gamma Q_{\omega}(s_{t+1}, a_{t+1}) - Q_{\omega}(s_t, a_t)] \nabla_{\omega} Q_{\omega}(s_t, a_t)$

$\delta$  TD error



**Problem: Instability**

Online Training: One experience at a time

Offline Training: In batches